
subhash maurya

Ashish Vaswani *
Google Cerveau
Avaswani@google.com

Noam Shazeer *
Google Cerveau
Noam@google.com

Niki Parmar *
Recherche Google
nikip@google.com

Jakob Uszkoreit *
Recherche Google
usz@google.com

Llion Jones *
Recherche Google
Illion@google.com

- Aidan N. Gomez *
Université de Toronto
Aiden@cs.toronto.edu

Kaiser *
Google Cerveau
lukaszkaizer@google.com

Illia Polosukhin *
Illia.polosukhin@gmail.com

Résumé

Les modèles dominants de transduction de séquence sont basés sur des réseaux neuronaux convolutionnels comprenant un codeur et un décodeur. Les réalisations de ces modèles sont basées sur des encodeurs et des décodeurs par un mécanisme d'attention. Nous proposons un nouveau simple réseau à architecture de Transformatrice, basé uniquement sur des mécanismes d'attention, dispensant avec récurrence et convolutions entières. Nos expériences de machine translation montrent que ces modèles atteignent une qualité supérieure tout en étant plus parallélisables et exigeant de manière significative moins de ressources. Notre modèle réalise un score de 28.4 UEBL sur la tâche de traduction de l'anglais vers l'allemand, améliorant ainsi les résultats existants de 1.0 point. Sur l'ensemble de la WMT 2014, notre modèle établit un nouveau score BLEU à la fine pointe de la technologie de 41.0 après formation de 3,5 jours sur huit GPU, une petite fraction des coûts de formation de nos prédécesseurs.

1 Présentation

Réseaux neuronaux récurrents, longue mémoire à court terme [12] et fermé récurrent [7] réseaux neuronaux en particulier, ont été fermement établis comme l'état de la technique approche dans la modélisation de séquence et problèmes de transduction tels que la modélisation linguistique et la traduction automatique [29, 2, 5]. Nombreux Depuis, les efforts se sont poursuivis pour repousser les limites des modèles de langage récurrents et des codeurs-décodeurs les architectures [31, 21, 13].

* Contribution égale. L'ordre d'inscription est : Jakob a proposé de remplacer les RNN par l'auto-attention et a commencé l'effort pour évaluer cette idée. Ashish, avec Illia, a conçu et mis en œuvre les premiers modèles Transformer et a joué un rôle crucial dans tous les aspects de ce travail. Noam a proposé l'attention sur les produits à points, multi-têtes l'attention et la représentation de position sans paramètre et est devenu l'autre personne impliquée dans presque chaque détails. Niki a conçu, implémenté, accordé et évalué d'innombrables variantes de modèles dans notre base de code originale et tensor2tensor. Llion a également expérimenté de nouvelles variantes de modèles, a été responsable de notre base de code initiale, et inférence et visualisations efficaces. Lukasz et Aidan ont passé d'innombrables longues journées à concevoir diverses parties de mettre en œuvre tensor2tensor, en remplaçant notre base de données précédente, en améliorant considérablement les résultats et en nous permettant de travailler plus efficacement.

- * Travail effectué à Google Brain.
- * Travaux effectués à Google Research.

Modèles récurrents calculent généralement les facteurs le long des positions symboliques de l'entrée et de la sortie séquences. Alignement des positions aux étapes du temps de calcul, elles génèrent une séquence de cache États h_t , en fonction de l'état caché précédent h_{t-1} et l'entrée pour la position t . Cette La nature séquentielle exclut la parallélisation dans les exemples de formation, qui devient critique à plus long terme longueurs de séquence, car les contraintes de mémoire limitent le batch entre les exemples. Travaux récents amélioration significative de l'efficacité de calcul par des astuces de factorisation [18] et conditionnelle calcul [26], tout en améliorant la performance du modèle dans le cas de ce dernier. Les Toutefois, la contrainte du calcul séquentiel demeure.

Les mécanismes d'attention font désormais partie intégrante de la modélisation et de la transduction des séquences. les modèles de travail dans diverses tâches, permettant de modéliser les dépendances sans tenir compte de leur distance en les séquences d'entrée ou de sortie [2, 16]. Dans tous les cas sauf [22], cependant, de tels mécanismes d'attention sont utilisés conjointement avec un réseau récurrent [23].

Dans ce travail, nous proposons le Transformer, une architecture modèle évitant la récurrence et à la place s'appuyant entièrement sur un mécanisme d'attention pour attirer les dépendances mondiales entre les intrants et les extrants. Le Transformateur permet une parallélisation significativement plus et peut atteindre un nouvel état de l'art dans qualité de traduction après avoir été formé pour aussi peu que douze heures sur huit P100 GPU.

2 Historique

L'objectif de réduire le calcul séquentiel forme également le fondement du GPU neuronal étendu [20], ByteNet [15] et ConvS2S [8], qui utilisent tous les réseaux neuronaux convolutionnels comme construction de bloc, calcul des représentations cachées en parallèle pour toutes les positions d'entrée et de sortie. Dans ces modèles, le nombre d'opérations nécessaires pour relier les signaux de deux positions d'entrée ou de sortie arbitraires augmente dans la distance entre les positions, linéairement pour ConvS2S et logarithmiquement pour ByteNet. Cela rend Ç plus difficile à apprendre dépendances entre lointain Fonctions [11]. D des Transformateur cet à un nombre constant d'opérations, mais au prix d'un règlement effectif réduit dû à une pondération de positions, un effet nous contraindre avec s Têtes multiples Attention te décrit à la section 3.

L'auto-attention, parfois appelée intra-attention, est un mécanisme d'attention reliant différentes positions d'une seule séquence afin de calculer une représentation de la séquence. L'auto-attention a été utilisé avec succès dans une variété de tâches, y compris la compréhension de la lecture, la synthèse abstraite, représentation des phrases indépendantes de la tâche [4, 22, 23, 19].

Les réseaux de mémoire de bout en bout sont basés sur un mécanisme d'attention récurrent au lieu de séquence-une récurrence alignée et il a été démontré qu'il y avait de bonnes performances en matière de réponse aux questions en tâche de compréhension linguistique [28]. Au des le de no connaissance, Toutefois, des Transformateur e des pre transduction modèle la entièrement mesur auto-attention pour calculer les représentations de ses entrées et sorties sans utiliser la séquence- confiance les RNN ou convolution alignées. Dans les sections suivantes, nous allons décrire le Transformateur, motiver e auto-attention et discuter de ses avantages par rapport à des modèles tels que [14, 15] et [8].

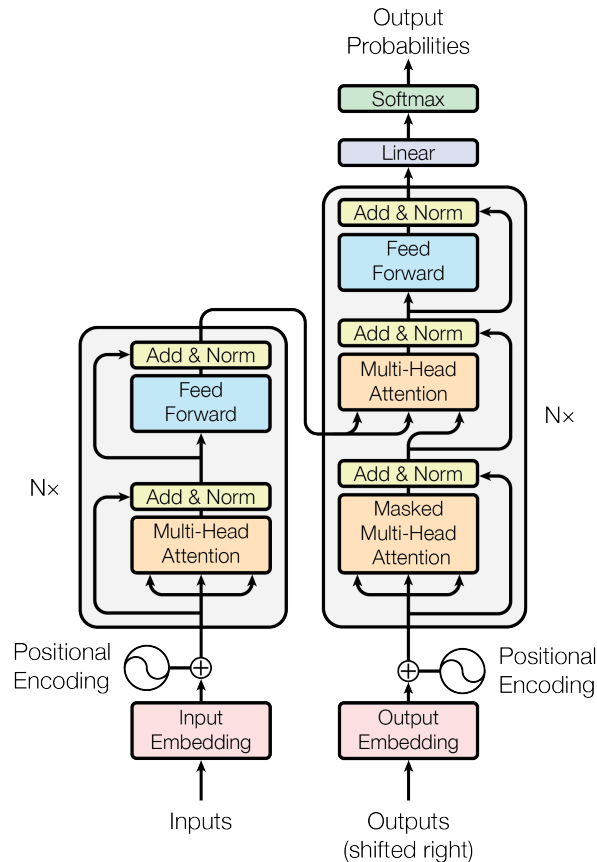
3 Architecture du modèle

La plupart des modèles concurrentiels de transduction des séquences neurales ont une structure encodeur-décodeur [29]. Ici, des encodeur cartes un entrée séquence de symbole représentations $(x_1, \dots, x_n)$ à a séquence ui de continu représentations $Z = (Z_1, \dots, z_{s_n})$. Donné Z , des décodeur puis génère un sortie séquence $(y_1, \dots, y_m)$ des symboles un élément à la fois. es A chaque étape, le modèle est auto-régressif [9], en consommant les symboles précédemment générés comme entrée supplémentaire lors de la génération suivante.

Le transformateur suit cette architecture globale en utilisant l'auto-attention empilée et pointue, entièrement couches connectées pour l'encodeur et le décodeur, indiquées dans les moitiés gauche et droite de la figure 1, respectivement.

3.1 Encodeur et décodeurs

Encodeur & Les encodeur e composé de a pile de $N = 6$ identique couches Chaq couch a deu 6 couches La première est un mécanisme d'auto-attention multi-tête, et la seconde est une simple position- x



Graphique 1: Le transformateur - architecture modèle.

réseau d'alimentation entièrement connecté. Nous employons une connexion résiduelle [10] autour de chaque des deux sous-couches suivie par une couche de normalisation. La fonction mise en œuvre par la sous-couche est $\text{LayerNorm}(x + \text{Sous-couche}(x))$, où $\text{Sous-couche}(x)$ est la fonction mise en œuvre par la sous-couche. Pour faciliter ces connexions résiduelles, toutes les sous-couches du modèle, ainsi que l'intégration des couches, produisent des sorties de dimension $d_{\text{modèle}} = 512$.

Décoder : Le décodeur est également composé d'une pile de $N = 6$ couches identiques. En plus des deux sous-couches dans chaque couche d'encodeur, le décodeur insère une troisième sous-couche, qui effectue plusieurs têtes d'attention sur la sortie de la pile encodeur. Comme pour l'encodeur, nous utilisons des connexions résiduelles autour de chacune des sous-couches, suivies d'une normalisation des couches. Nous modifions aussi l'auto-attention des sous-couches du décodeur pour empêcher les têtes d'attention de participer à des positions futures. Cette modification est combinée avec le fait que les intégrations de sortie sont compensées par une position, de sorte que la prédiction pour la position i ne peut dépendre que des sorties connues à des positions inférieures à i .

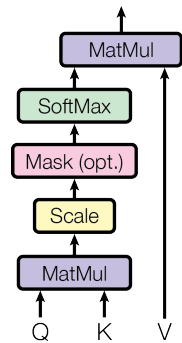
3.2 Attention

Une fonction d'attention peut être décrite comme un mappage d'une requête et d'un ensemble de paires de valeurs clés à une sortie. La requête, les clés, les valeurs et la sortie sont tous des vecteurs. La production est calculée comme une somme des valeurs, lorsque le poids attribué à chaque valeur est calculé par une pondération de compatibilité de la requête avec la clé correspondante.

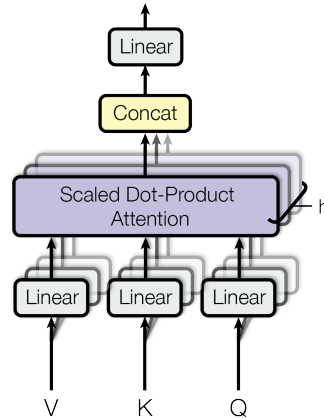
3.2.1 Attention au produit à point d'échelle

Nous attirons notre attention particulière sur l'attention accordée aux produits à point réduit (figure 3.2.1). L'entrée se compose de requêtes et clés de dimension d_k , et valeurs de dimension d_v . Nous calculons les produits à point de

Attention au produit à point d'échelle



Attention à plusieurs têtes



Graphique 2 (à gauche) : Attention au produit à point d'échelle. (à droite) : Attention à plusieurs têtes. Le graphique de gauche illustre le processus d'attention au produit à point d'échelle, tandis que le graphique de droite illustre l'attention à plusieurs têtes.

requête avec toutes les clés, diviser chaque résultat par $\sqrt{d_k}$, et appliquer une fonction softmax pour obtenir les poids sur les valeurs.

Dans la pratique, nous calculons la fonction d'attention sur un ensemble de requêtes simultanément, dans une matrice Q . Les clés et les valeurs sont aussi emballées dans des matrices K et V . Nous calculons la matrice des résultats comme suit :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

Les deux fonctions d'attention les plus couramment utilisées sont l'attention additive [2], et produit à point (multi-attention).

L'attention du produit dot est identique à notre algorithme, sauf pour le facteur d'échelle de $\frac{1}{\sqrt{d_k}}$. Additive attention calcule la fonction de compatibilité à l'aide d'un réseau feed-forward avec une seule couche cachée.

Alors que les deux sont similaires dans la complexité théorique, l'attention produit point est beaucoup plus rapide et plus spatialement efficace dans la pratique, car il peut être mis en œuvre en utilisant hautement optimisé code de multiplication de matrice.

Alors que pour de petites valeurs de d_k , les deux mécanismes fonctionnent de la même manière, l'attention additive dépasse les performances de l'attention produit sans échelle pour des valeurs plus grandes de d_k [3]. Nous soupçonnons que pour de grandes valeurs de d_k , les produits à points grandissent, poussant la fonction de compatibilité à des valeurs de $\frac{1}{\sqrt{d_k}}$ qui deviennent extrêmement petits.

3.2.2 Attention à plusieurs têtes

Au lieu d'effectuer une seule fonction d'attention avec les clés, valeurs et requêtes, nous avons trouvé utile de projeter linéairement les requêtes, les clés et les valeurs en d_k , d_k et d_v dimensions, respectivement. Sur chacune de ces versions projetées, nous effectuons la fonction d'attention en parallèle, ce qui donne des valeurs de dimension d_v . Ces valeurs sont concaténées et encore projetées, résultant en des valeurs de dimension d_v . Figure 2.

L'attention multi-têtes permet au modèle d'assister conjointement à des informations provenant de différentes représentations sous-espaces à différentes positions. Avec une seule tête d'attention, la moyenne inhibe cela.

⁴Pour illustrer pourquoi les produits à points deviennent grands, supposons que les composants q et k sont indépendants au hasard. Puis leur produit point, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, a une moyenne de 0 et une variance de d_k .

$$\text{Têtes multiples } (Q, K, V) = \text{Concat}(\text{tête}_1 \dots \text{tête}_h) W^O$$

$$\text{où } \text{tête}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Lorsque les projections sont des matrices de paramètres $W_i^Q * P_{\text{modèle}}^d, W_i^K * P_{\text{modèle}}^d, W_i^V * P_{\text{modèle}}^d$ et $W^O * P_{\text{modèle}}^d$, nous avons $d_{\text{modèle}} = 512$. En raison de la dimension réduite de chaque tête, le coût total des calculs est similaire à celui d'une seule tête avec la pleine dimensionalité.

3.2.3 Applications de l'attention dans notre modèle

Le Transformateur utilise l'attention multi-têtes de trois façons différentes:

- 1. Dans les calques "encoder-décodeur attention", les requêtes proviennent du calque de décodeur précédent, et les clés et valeurs de mémoire proviennent de la sortie de l'encodeur. Cela permet à chaque position dans le décodeur pour assister à toutes les positions dans la séquence d'entrée. Cela imite les modèles de type codeur-décodeur attention mécanismes da séquence à séquence tels que [31, 2, 8].
- 2. L'encodeur contient des couches d'auto-attention. Dans une couche auto-attention toutes les clés, valeurs et les requêtes viennent du même endroit, dans ce cas, la sortie de la couche précédente dans le Encodeur. Chaque position de l'encodeur peut s'occuper de toutes les positions de la couche précédente Encodeur.
- 3. De même, les couches d'auto-attention dans le décodeur permettent à chaque position du décodeur de s'occuper de toutes les positions dans le décodeur jusqu'à cette position. Nous devons empêcher la gauche flux d'information dans le décodeur pour préserver la propriété auto-régressive. Nous mettons en ?uvre à l'intérieur de l'attention du produit à points éparpillés en masquant ? toutes les valeurs dans du softmax qui correspond à des connexions illégales. Voir figure 2. l'entrée

3.3 Réseaux d'alimentation en fonction de la position

En plus des sous-couches d'attention, chacune des couches de notre codeur et décodeur contient un réseau d'alimentation-avant connecté, qui est appliqué à chaque position séparément et de manière identique. Cette se compose de deux transformations linéaires avec une activation ReLU entre les deux.

$$FN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

Alors que les transformations linéaires sont les mêmes pour différentes positions, elles utilisent différents paramètres de couche à couche. Un autre Voie de décrivant cet e co deu convolutions avec noyau taille 1. Les dimensionnalité de entrée et sortie e d modèle = 512, met x des couche a dimensionnalité d ff = 2048 . s st modèle interne

3.4 Embeddings et Softmax

De même que d'autres modèles de transduction de séquence, nous utilisons des incrustations apprises pour convertir l'entrée jetons et jetons de sortie aux vecteurs de dimension $d_{\text{modèle}}$. Nous utilisons aussi la transfor- linéaire apprise mation et la fonction softmax pour convertir la sortie du décodeur en probabilités prévues pour le prochain décodeur. Dans notre modèle, nous partageons la même matrice de poids entre les deux couches d'intégration et le pré-softmax ? an transformation linéaire, similaire à [24]. Dans les couches d'intégration, nous multiplions ces poids par $d_{\text{modèle}}$ ui

3.5 Encodage positionnel

Comme notre modèle ne contient aucune récurrence et aucune convolution, afin que le modèle puisse utiliser le ordre de la séquence, nous devons injecter quelques informations sur la position relative ou absolue de la Jetons dans la séquence. À cette fin, nous ajoutons des "encodages positionnels" à l'encodage d'entrée

Tableau 1 Longueurs maximales du trajet, complexité par couche et nombre minimal d'opérations séquentielles pour différents types de couches convolutions et récurrentes

n est la longueur de la séquence, d est la dimension de représentation, k est le noyau de convolution, r est la taille du quartier dans l'auto-attention restreinte.

Type de couche	Complexité par couche	Séquentiel Opérations	Longueur maximale du sentier
L'auto-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Récurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutionnel	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\text{jour}_k(n))$
Auto-attention (restriction)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

les fonds des piles encodeur et décodeur. Les encodages positionnels ont la même dimension $d_{\text{modèle}}$ comme les emboîtements, afin que les deux puissent être combinés. Il existe de nombreux choix d'encodages positionnels, révisés et corrigés [8].

Dans ce travail, nous utilisons des fonctions sinus et cosinus de fréquences différentes:

$$PE_{(\text{oui}, 2i)} = \sin(\text{os} / 10000^{2i/d_{\text{modèle}}})$$

$$PE_{(\text{oui}, 2i+1)} = \cos(\text{os} / 10000^{2i/d_{\text{modèle}}})$$

où os est la position et i est la dimension. Autrement dit, chaque dimension de l'encodage positionnel correspond à un sinus. Les longueurs d'onde forment une progression géométrique de 2^2 à $10000 \cdot 2^2$. Nous avons choisi cette fonction parce que nous avons supposé qu'elle permettrait au modèle d'apprendre facilement à assister à des postes relatifs, depuis pour tout décalage fixe k , $PE_{\text{os} + k}$ peut être représenté comme une fonction linéaire de PE_{os} .

Nous avons également expérimenté l'utilisation d'intégrations positionnelles apprises [8] plutôt, et a trouvé que les deux versions ont produit des résultats presque identiques (voir tableau 3 ligne (E)). Nous avons choisi la version sinusoïdale car il peut permettre au modèle d'extrapoler des longueurs de séquence plus longues que celles rencontrées pendant l'entraînement.

4 Pourquoi l'auto-attention

Dans cette rubrique nous comparons divers aspects de l'auto-attention à des couches à connexion périodique et à des couches à connexion couramment utilisées pour la cartographie d'une séquence de symboles de longueur variable à l'essentiel, $(x_1, \dots, x_n)$ à une autre séquence de même longueur $(z_1, \dots, z_n)$, avec $x_i = P_i^d$, tels qu'ils sont cachés dans une couche d'encodeur ou d'un décodeur de séquence typique. Motivant notre utilisation de l'auto-attention envisager trois desiderata.

Un autre est le montant de calcul qui peut être parallélisé, selon le nombre minimum d'opérations séquentielles requises.

Le troisième est la longueur du chemin entre les dépendances à longue distance dans le réseau. Apprendre à long terme Les dépendances sont un défi clé dans de nombreuses tâches de transduction de données. Un facteur clé ayant une incidence sévère sur l'apprentissage de telles dépendances est la longueur des chemins vers l'avant et les signaux vers l'arrière doivent traverser dans le réseau. Plus ces chemins sont courts entre n'importe quelle combinaison de positions dans l'entrée et les séquences de sortie, plus il est facile d'apprendre les dépendances à long terme [11]. Par conséquent, nous la longueur maximale du chemin entre deux positions d'entrée et de sortie dans les réseaux complexes nous permet également de comparer différents types de couches.

Comme indiqué au tableau 1, une couche d'auto-attention relie toutes les positions avec un nombre constant de séquences exécutées opérations, tandis qu'une couche à connexion périodique nécessite $O(n)$ séquentiel les termes de complexité informatique, les couches auto-attention sont plus rapides que les couches récurrentes pour la séquence longue n est plus grande que la dimensionnalité d , qui est la souvenance. Cas avec des représentations de phrases utilisées par les modèles les plus modernes dans les traductions automatiques, comme les [10] et [25] représentations. Améliorer les performances de calcul pour les tâches impliquant très longues séquences, l'auto-attention pourrait être limitée à considérer seulement un quartier de taille r dans

la séquence d'entrée centrée autour de la position de sortie respective. Cela augmenterait le maximum longueur du chemin jusqu'à une seule couche convolutionnelle avec largeur du noyau $k < n$ ne connecte pas toutes les paires d'entrée et de sortie positions. Le faire nécessite une pile de $O(n/k)$ couches convolutionnelles dans le cas de grains contigus, ou $O(n/k)$ dans le cas de convolutions dilatées [15], augmenter la longueur des sentiers les plus longs entre deux positions du réseau. Les couches convolutionnelles sont généralement plus chères que les couches attentionnelles. Toutefois, la diminution de complexité considérable pour les convolutions séparables ($k \cdot n \cdot d + n \cdot d^2$). Même avec $k = n$, Toutefois, la complexité d'un séparable la convolution est égale à la combinaison d'une couche d'auto-attention et d'une couche d'alimentation ponctuelle, l'approche que nous adoptons dans notre modèle. En tant qu'avantage secondaire, l'auto-attention pourrait donner des modèles plus interprétables. Nous inspectons les distributions d'attention pour présenter et discuter des exemples dans l'annexe. Non seulement l'attention individuelle têtes clairement apprendre à effectuer des tâches différentes, beaucoup semblent montrer un comportement lié à la syntaxique et la structure sémantique des phrases.

5 Formation

Cette section décrit le régime de formation de nos modèles.

5.1 Données de formation et abattage

Nous formés le des standard WMT 2014 Anglais-allemand ensembl comprenant de sur 4,5 millions Les paires de Les peines ont été encodées en utilisant l'encodage byte-pair [3], qui a une source partagée pour le français-anglais. Pour le français-anglais, nous avons utilisé le WMT beaucoup plus grand Ensemble de données français-anglais 2014 composé de 36M phrases et jetons divisés en 32000 mots vocabulaire [31]. Les paires de peines ont été groupées par une longueur de séquence approximative. Chaque lot contenait un ensemble de paires de phrases contenant environ 25000 jetons source et 25000 jetons cibles.

5.2 Matériel et calendrier

Nous avons formé nos modèles sur une seule machine avec 8 processeurs NVIDIA P100. Pour nos modèles de base les hyperparamètres décrits dans l'article, chaque étape d'entraînement a pris environ 0,4 seconde. Nous formation des modèles de base pour un total de 100 000 étapes ou 12 heures. Pour nos grands modèles, s ligne du bas du tableau 3), le temps d'étape était de 1.0 Les grands modèles ont été formés pour 300 000 étapes (300000).

5.3 Optimisateur

Nous avons utilisé l'optimiseur Adam [17]. Avec $\beta_1 = 0.9$, $\beta_2 = 0.98$ et $\epsilon = 10^{-9}$. Nous avons varié le taux sur le cours de formation, selon la formule: l'apprentissage

$$\text{Taux}_{\text{modèle}} = d \cdot 0.5 \cdot \min(\text{étape}_{\text{num}} \cdot 0.5, \text{étape}_{\text{num}} \cdot \text{échauffement} \cdot \text{étapes}^{-1.5}) \quad (3)$$

Cela correspond à une augmentation linéaire du taux d'apprentissage pour la première échauffement étapes les étapes et la diminuer ensuite proportionnellement à la racine carrée inverse du nombre d'étapes. Nous avons utilisé $\text{échauffement} \cdot \text{étapes} = 4000$.

5.4 Régularisation

Nous employons trois types de régularisation pendant la formation :

Décrochage résiduel Nous postulons [27] à la sortie de chaque sous-couche, avant qu'elle ne soit ajoutée au sous-couche entrée et normale. En outre, nous appliquons l'abandon scolaire aux sommes des les encodages positionnels dans les piles de codeur et de décodeur. Pour le modèle de base, nous utilisons un taux $P_{\text{chute}} = 0.1$.

Tableau 2 Le transformateur obtient de meilleurs scores BLEU que les précédents modèles de pointe sur le test de nouvelles de l'anglais vers l'allemand et de l'anglais vers l'anglais2014 à une fraction du coût de la formation.

Modèle	UEBL		Coût de la formation	
	EN-DE	EN-FR	EN-DE	EN-FR
BYTENET [15]	23.75			
Attaque profonde + Posunk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MdE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
Ensemble GNMT + RL [31]	26 h	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	30.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformateur (modèle de base)	27,3	38,1	$3.3 \cdot 10^{18}$	
Transformateur (grand)	28.4	41,0	$2.3 \cdot 10^{19}$	

Lissage des étiquettes Au cours de la formation, nous avons employé le lissage des étiquettes $p_e = 0.1$ [30]. Cette technique permet de contrôler la perplexité, car le modèle apprend à être plus incertain, mais améliore la précision et le score BLEU.

6 Résultats

6.1 Traduction automatique

Sur la tâche de traduction de l'anglais à l'allemand de WMT 2014, le grand modèle de transformateur (transformateur (grand) dans le tableau 2) surpasse de plus de 28.4 points de BLEU, établissant un nouveau score BLEU à la fine pointe de la technologie. La configuration de ce modèle est décrite dans le tableau 3. Formation 3.5 jours 8 P100 GPU. Même notre modèle de base surpasse tous les modèles et ensembles publiés précédemment, à une fraction du coût de la formation d'un des les modèles compétitifs.

Sur la tâche de traduction de l'anglais au français de la WMT 2014, notre grand modèle obtient un score BLEU de 41.0, surperformant tous les modèles uniques publiés précédemment, à moins de 1/4 le coût de la formation précédent modèle de pointe. Le modèle du transformateur (grand) formé pour l'anglais au français utilisé Taux d'abandon $P_{chute} = 0.1$, au lieu de 0.3. Pour les modèles de base, nous avons utilisé un modèle unique obtenu en calculant la moyenne des 5 derniers points de contrôle à intervalles de 10 minutes. Pour les grands modèles, nous avons fait la moyenne des 20 derniers points de contrôle. Ces hyperparamètres ont été choisis après expérimentation sur l'ensemble de développement. Nous avons défini la longueur maximale de sortie avec une taille de faisceau de 50, mais se terminer plus tôt si possible pendant l'inférence.

Le tableau 2 résume nos résultats et compare nos coûts de traduction et de formation à d'autres modèles des architectures de la littérature. Nous estimons le nombre d'opérations flottantes utilisées pour former un modèle en multipliant le temps de formation, le nombre de GPU utilisés, et une estimation du capacité flottante à une seule précision de chaque GPU.

6.2 Variations du modèle

Pour évaluer l'importance des différentes composantes du Transformateur, nous avons varié notre modèle de base de différentes manières, en mesurant le changement de performance sur la traduction de l'anglais à l'allemand sur la ensemble de développement. Nous avons utilisé la recherche de faisceau comme décrit dans la section 3.2.2. Nous présentons nos résultats dans le tableau 3. Dans le tableau 3 lignes (A), nous varions le nombre de têtes d'attention et les dimensions de la clé d'attention et de la valeur, garder des Montant de calcul constante, ce que décrit le Chapitre 3.2.2. Alors Une seule tête d'attention est de 0,9 BLEU pire que le meilleur réglage, la qualité baisse également avec trop de têtes.

⁵Nous avons utilisé des valeurs de 2,8, 3,7, 6,0 et 9,5 TFLOPS pour K80, K40, M40 et P100, respectivement.

Tableau 3 Variations sur l'architecture Transformer. Les valeurs non cotées sont identiques à celles de la base de données. Toutes les mesures sont sur l'ensemble de développement de traduction de l'anglais à l'allemand, Inscrit dans le répertoire de la Bibliothèque nationale de France, selon notre codage byte-pair, et ne doivent pas être comparées à Perplexités par mot.

	N	$d_{\text{modèle}}$	d_{ff}	h	d_k	d_v	P_{chute}	η_e	train étape	PPL (dev)	UEBL (dev)	params A 10^6
base	6	512	2048	8	64	64	0,1	0,1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	n ex e
				4	128	128				5,00	25,5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23,7	36
	4									5.19	25.3	50
	8									4.88	25,5	80
		256			32	32				5.75	24,5	28
		1024			128	128				4.66	26,0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0,0			5.77	24,6	
							0,2			4.95	25,5	
								0,0		4.67	25.3	
								0,2		5.47	25,7	
(E)									emboîtement positionnel au lieu des sinusoïdes	4.92	25,7	
gros	6	1024	4096	16			0,3		300K	4,33	26,4	213

Dans le tableau 3 rangées (B), nous observons que la réduction de la taille de la clé d_k Ça nuit à la qualité du Cette
d'attention que déterminant compatibilité e pas facile et que a plus modèle, compatibilité
fonction que le produit point peut être bénéfique. st Nous observons en outre dans les lignes (C) et (D) que, comme prévu,
Les modèles plus grands sont meilleurs, et l'abandon est très utile pour éviter le surajustement. À la ligne (E), nous remplaçons
encodage positionnel sinusoïdal avec des encodages positionnels appris [8], et observer presque identique
résultats au modèle de base.

7 Conclusion

Dans ce travail, nous avons présenté le Transformer, le premier modèle de transduction de séquence basé entièrement sur
attention, en remplaçant les couches récurrentes les plus couramment utilisées dans les architectures encoder-decoder par
l'auto-attention multi-tête.

Pour les tâches de le transformateur peut être formé considérablement plus rapidement que les architectures basées
traduction ou convolutionnel couches À les WMT 2014 Anglais à allemand et WMT 2014
Tâches de traduction de l'anglais au français, nous atteignons un nouvel état de l'art. Dans l'ancienne tâche, notre
modèle surpasse même tous les ensembles précédemment signalés. meilleur

Nous sommes ravis de l'avenir des modèles axés sur l'attention et nous prévoyons les appliquer à d'autres tâches. Nous
projet d'étendre le Transformateur aux problèmes concernant les modalités d'entrée et de sortie autres que le texte et s
d'étudier les mécanismes locaux d'attention restreinte pour gérer efficacement les gros intrants et produits
tels que les images, audio et vidéo. Rendre la génération moins séquentielle est un autre objectif de recherche de notre

Les code nou utili à train et évaluer no modèles e disponibles à <https://github.com/tensorflow/tensor2tensor>. se tre st

Remerciements Nous sommes reconnaissants à Nal Kalchbrenner et Stephan Gouws pour leur
commentaires, corrections et inspiration.

Références

- An Jimmy Lei Ba, Jamie Ryan Kiros et Geoffrey E Hinton. Normalisation des préimpression
nex arXiv:1607.06450 , 2016. couches. arXiv
e
- [2] Dzmitry Bahdanau, Kyunghyun Cho et Yoshua Bengio. Traduction automatique neuronale en commun
apprendre à aligner et à traduire. CoRR , abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong et Quoc V. Le. Exploration massive de neurones
architectures de traduction automatique. CoRR , abs/1703.03906, 2017.
- An Jianpeng Cheng, Li Dong et Mirella Lapata. Longs réseaux de mémoire à court terme pour machine
nex lecture. arXiv préimpression arXiv:1601.06733 , 2016.
e
- [5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk,
et Yoshua Bengio. Apprentissage des représentations de phrases en utilisant rnn encoder-decoder pour
traduction automatique. CoRR , abs/1406.1078, 2014.
- [6] François Chouette. Xception : profond appren avec profondeur séparable les convolutions. arXiv
préimpression arXiv:1610.02357 , 2016. tissage
- [10] Junyoung Chung, Caglar Gülçehre, Kyunghyun Cho et Yoshua Bengio. Évaluation empirique
des réseaux neuraux récurrents fermés sur la modélisation des CoRR , abs/1412.3555, 2014.
séquences.
- [10] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats et Yann N. Dauphin. Convolu...
l'apprentissage séquentiel. arXiv préimpression arXiv:1705.03122v2 , 2017.
- [10] Alex Graves. Génération séquences avec périodique neural les réseaux. arXiv préimp
arXiv:1308.0850 , 2013. resion
- [10] Kaiming Lui, Xiangyu Zhang, Le -Oui. et Jian Sol profond résiduel appren pou Les
âge reconnaissance D Délibérations Shanghai IEEE Conférence le d'ordinateur Vision et Modèle
Reconnaissance, pages 770 à 778, 2016. s tissage
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi et Jürgen Schmidhuber. Débit en
les filets la difficulté d'apprendre les dépendances à long terme, 2001.
- [12] Sepp Hochreiter et Jürgen Schmidhuber. Longu court terme mémoire. Neural calcul ,
9(8):1735-1780, 1997. e
- [13] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer et Yonghui Wu. Exploration
les limites de la modélisation des arXiv préimpression arXiv:1602.02410 , 2016.
langues.
- [14] Kaiser et Ilya Sutskever. Les GPU neuraux apprennent les D Conférence internationale
sur les représentations éducatives (ICLR) algorithmes. an
s
- [15] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves et Ko-
Ray Kavukcuoglu. Traduction automatique neuronale dans le temps arXiv préimpression arXiv:1610.10099v2 ,
2016. préimpression
- [16] Yoon Kim, Carl Denton, Luong Hoang et Alexander M. Dépêche-toi. Réseaux d'attention structurés.
D Conférence internationale sur les représentations en matière d ' , 2017.
anapprentissage
- [17] Diederik Kingma et Jimmy Ba. Adam : Une méthode d'optimisation stochastique. D Groupe, 2015.
ande
- [18] Oleksii Kuchaiev et Boris Ginsburg. Trucs de factorisation pour les réseaux LSTM. préimpression
arXiv: 1703.10722 , 2017. arXiv
uverte
- [19] Zhouhan - Minwei Feng, Cicéron Nogueira dos Santos, Pay Oui. Bing Xiang et Coucou
Zhou, et Yoshua Bengio. A structuré auto-assistance phrase s L'intégration. arXiv préimp
arXiv: 1703.03130 , 2017. resion
- [20] Samy Bengio ?ukasz Kaiser. La mémoire active peut-elle remplacer l'attention D Progrès réalisés en
Systèmes de traitement de l'information (NIPS) , 2016. anneurologie
s

- [21] Minh-Thang Luong, Hieu Pham et Christopher D Manning. Approches efficaces de l'attention traduction automatique neurale basée. arXiv préimpression arXiv:1508.04025 , 2015.
- [22] Ankur Parikh, Oscar Täckström, Dipanjan Das et Jakob Uszkoreit. Une attention décomposable modèle. D Méthodes empiriques dans le traitement des langues naturelles , 2016.
- [23] Romain Pégulus, Caiming Xiong et Richard Socher. Un modèle profondément renforcé pour Résumé. arXiv préimpression arXiv:1705.04304 , 2017, abstraction
- [24] Ofir Press et Lior Wolf. Utilisation de l'intégration de sortie pour améliorer les modèles de langage. arXiv avant impression , 2016. arXiv:1608.05859
- [25] Rico Sennrich, Barry Haddow et Alexandra Birch. Traduction automatique neuronale de mots rares avec des sous-mots. arXiv préimpression arXiv:1508.07909 , 2015.
- [26] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, et Jeff Dean. Réseaux neuronaux de grande envergure: Le mélange d'experts peu avancé couch arXiv préimpression arXiv: 1701.06538 , 2017.
- [27] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever et Ruslan Salakhutdinov. Abandon : un moyen simple d'éviter que les réseaux neuronaux ne s'adaptent trop. Journal de la Machine Recherche en , 15(1):1929-1958, 2014.
- [28] Sainbayar Soukhbaatar, Arthur Szlam, Jason Weston, et Rob Fergus. De bout en bout en mémoire les réseaux. En C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, et R. Garnett, éditeurs, Progrès dans les systèmes de traitement de l'information neuronale 28 , pages 2440 à 2448. Curran Associés, Inc., 2015.
- [29] Ilya Sutskever, Oriol Vinyals et Quoc VV Le. Séquence à séquencer l'apprentissage avec neurone les réseaux. D Progrès dans les systèmes de traitement de l'information neuronale , pages 3104-3112, 2014.
- [30] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonas Shlens et Zbigniew Wojna. Repenser l'architecture de départ pour la vision informatique. CoRR , abs/1512.00567, 2015.
- [31] Yonghui Wu, C'est Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Machine neurale de Google système de traduction: Comble l'écart entre traduction humaine et traduction automatique. préimpression arXiv:1609.08144 2016. arXiv
- [32] Jie Zhou, Ying Cao, Xuguang Wang, Peng - et Wei Ouy. profond périodique modèles avec connexions rapides pour la traduction automatique neuronale. CoRR , abs/1606.04199, 2016.